

DataShare

Documentation Technique Complète

Projet 4 — OpenClassrooms, parcours Architecte logiciel
Mission « Pilotez le développement d'une solution informatique »

Dépôt : <https://github.com/GadZouil/OC-P4-DataShare>

Ethan Legros — Référent technique senior, DataShare

Septembre 2026

Table des matières

1	Architecture de l'application	4
1.1	Architecture générale	4
1.2	Structure du projet	4
1.3	Patterns utilisés	6
2	Choix technologiques justifiés	7
2.1	Backend	7
2.2	Tests Backend	8
2.3	Frontend	9
2.4	Infrastructure	9
3	Modèle de données	9
3.1	Entité AppUser	9
3.2	Entité FileItem	10
3.3	Relations	11
3.4	Configuration Fluent API	11
3.5	Historique des migrations	11
4	Documentation d'API	12
4.1	Auth — /api/auth	12
4.2	Files (authentifié) — /api/files	12
4.3	Liens publics — /api/public/files	14
5	Sécurité et gestion des accès	15
5.1	Authentification JWT	15
5.2	Hachage des mots de passe	17
5.3	Validation des entrées	17
5.4	Protection des fichiers	17
5.5	Anti-injection SQL	17
5.6	CORS	17
5.7	Transport et journalisation	18
5.8	Gestion des secrets	18
5.9	Améliorations futures	18
6	Qualité, tests et maintenance	18
6.1	Stratégie de tests	18
6.2	Plan de tests	19
6.3	Couverture de code	20
6.4	Performance	20
6.4.1	Budget de performance	20
6.4.2	Résultats du test de charge k6	20
6.4.3	Logs structurés et métriques clés	21
6.4.4	Budget frontend (Lighthouse)	21
6.4.5	Résultats Lighthouse (audit du 31/03/2026)	22
6.5	Maintenance	22
6.5.1	Gestion des services (Docker)	22

6.5.2	Sauvegarde et restauration de la base	23
6.5.3	Mise à jour des dépendances	23
6.5.4	Procédure de correction de bugs	23
6.5.5	Nettoyage des fichiers expirés	23
6.5.6	Checklist de mise en production	23
7	Processus d'installation et d'exécution	24
7.1	Prérequis	24
7.2	Installation manuelle	24
7.3	Installation Docker (tout-en-un)	25
7.4	Variables d'environnement	25
8	Utilisation de l'IA dans le développement	26
8.1	Posture adoptée	26
8.2	Tâche principale : US08 — Gestion des tags (frontend)	27
8.2.1	Contexte	27
8.2.2	Fichiers modifiés par l'IA	27
8.3	Supervision et corrections humaines	27
8.4	Autres usages de l'IA	28
8.5	Apports et limites constatés	28
8.5.1	Apports	28
8.5.2	Limites	28
8.6	Conclusion sur l'utilisation de l'IA	29

1 Architecture de l'application

1.1 Architecture générale

DataShare adopte une architecture en **trois couches** inspirée des principes Clean Architecture, où chaque couche a une responsabilité clairement définie :

- **API Layer** — Contrôleurs ASP.NET Core (`AuthController`, `FilesController`, `PublicFilesController`) qui exposent les endpoints REST. Les DTOs (records et classes) sont définis directement dans les contrôleurs pour limiter la complexité.
- **Service Layer** — Logique métier encapsulée dans des services injectés : `IFileStorage` / `LocalFileStorage` (stockage fichiers), `ExpiredFilesCleanupService` (purge automatique), et ASP.NET Core Identity (`userManager<AppUser>`) pour la gestion des utilisateurs.
- **Data Layer** — Entity Framework Core 9 avec le `DataShareDbContext` (héritant de `IdentityDbContext`) communiquant avec PostgreSQL 16 via le provider Npgsql.

Le **frontend** est une Single Page Application Vue 3 (Composition API + TypeScript) servie par Vite, qui consomme l'API REST via Axios.

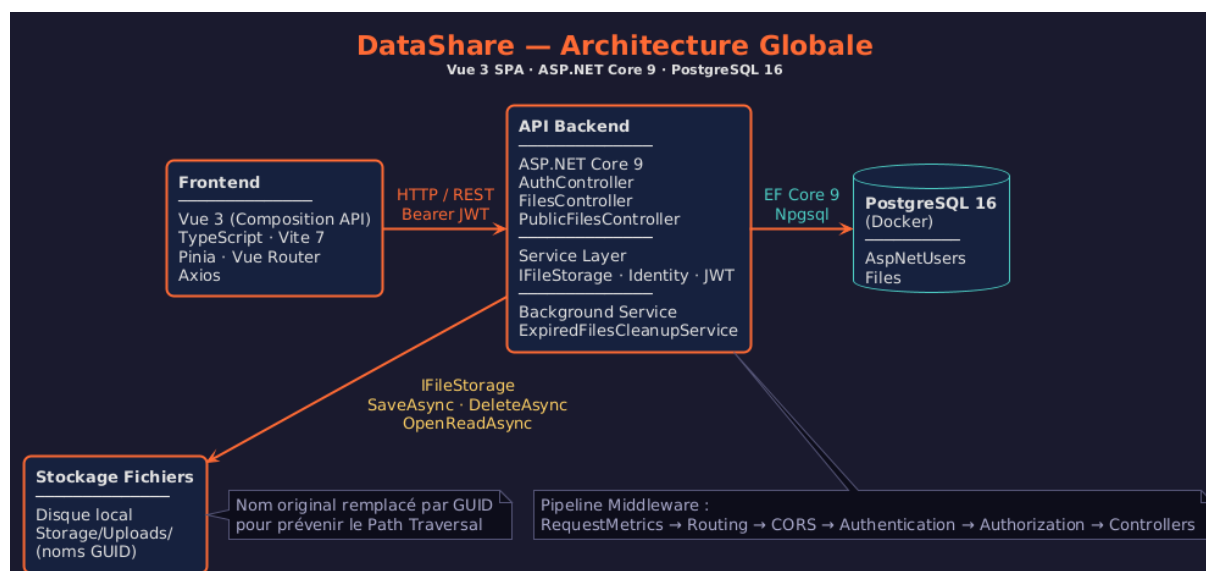


FIGURE 1 – Architecture globale de DataShare — Vue 3 SPA, API ASP.NET Core 9, PostgreSQL 16 et stockage fichiers

1.2 Structure du projet

Arborescence du projet

```
OC - P4 - DataShare /
  backend /
    DataShare.Api /
      Controllers /          # AuthController, FilesController,
      PublicFilesController
      Data /                  # DataShareDbContext
      Middleware /            # RequestMetricsMiddleware (logs
      structures par requete)
```

```

Migrations/           # 4 migrations EF Core
Models/               # AppUser, FileItem
Services/             # IFileStorage, LocalFileStorage,
    ExpiredFilesCleanupService
Program.cs            # Point d'entree, DI, middleware pipeline
appsettings.json      # Configuration (connexion DB, JWT, CORS,
    logs)
DataShare.Api.csproj
DataShare.Api.Tests/
    Helpers/          # CustomWebApplicationFactory,
        FakeFileStorage, TestAuthHandler
    IntegrationTests/ # 10 fichiers, 39 tests d'integration
    UnitTests/        # ExpiredFilesCleanupServiceTests (2
        tests)
DataShare.sln
Dockerfile            # Build .NET 9 multi-stage
frontend/
    datashare-front/
        src/
            api/       # auth.ts, files.ts (clients HTTP)
            layouts/   # PublicLayout.vue
            router/    # index.ts (5 routes + garde d'
                authentification)
            services/  # api.ts (instance Axios : JWT, gestion
                du 401)
            stores/    # auth.ts (Pinia)
            styles/    # datashare.css (charte Figma)
            views/     # 5 vues : Login, Register, Upload, Me,
                Download
            App.vue, main.ts
            vite.config.ts
            package.json
        cypress/
            e2e/        # 5 fichiers de tests E2E
            support/    # commands.ts, e2e.ts
        Dockerfile     # Node 20 + Nginx
docs/
    diagrams/          # architecture, MCD, flux (PlantUML + PNG
        , draw.io)
    API_REFERENCE.md   # Documentation API complete
    openapi.yaml       # Contrat d'interface OpenAPI 3
    soutenance/        # Support de presentation, script oral,
        scenario de demo
perf/                 # Scripts k6 (upload, login)
scripts/              # deploy.ps1/.sh, db-backup.ps1, db-
    restore.ps1, verify.ps1
docker-compose.yml    # 3 services : db, api, frontend (secrets
    lus dans .env)
.env.example
README.md, SECURITY.md, TESTING.md, PERF.md, MAINTENANCE.md,
AI_USAGE.md

```

1.3 Patterns utilisés

Dependency Injection (DI)

Le conteneur IoC natif d'ASP.NET Core est utilisé intensivement. Dans `Program.cs`, les services sont enregistrés via `AddSingleton<IFileStorage, LocalFileStorage>()` et `AddIdentityCore<AppUser>()`. Les contrôleurs reçoivent leurs dépendances par injection de constructeur (`DataShareDbContext, IFileStorage, UserManager<AppUser>`).

Interface Segregation (IFileStorage)

Le stockage de fichiers est abstrait derrière l'interface `IFileStorage` définissant trois opérations : `SaveAsync`, `DeleteAsync`, `OpenReadAsync`. L'implémentation `LocalFileStorage` stocke les fichiers sur le système de fichiers local dans `Storage/Uploads/`. Cette abstraction permet de remplacer le stockage par un provider cloud (Azure Blob, S3) sans modifier les contrôleurs.

DTOs (Data Transfer Objects)

Les records C# (`MyFileDto`, `AuthResponse`, `RegisterRequest`, `LoginRequest`) et classes (`UploadRequest`, `DownloadRequest`) sont définis dans les contrôleurs pour découpler le modèle de domaine des réponses API. Les propriétés calculées (`IsExpired`, `IsPasswordProtected`) du modèle ne sont pas exposées directement.

Background Service

`ExpiredFilesCleanupService` hérite de `BackgroundService` et s'exécute automatiquement au démarrage puis toutes les 24 heures via un `PeriodicTimer`. Il purge les fichiers expirés (stockage disque + entrées en base) en utilisant un scope de DI dédié (`IServiceScopeFactory`).

Middleware Pipeline

Le pipeline HTTP est configuré dans `Program.cs` avec l'ordre : `UseRequestMetrics` (journalisation structurée de chaque requête) → `UseRouting` → `UseCors` → `UseAuthentication` → `UseAuthorization` → `MapControllers`. Le middleware de métriques est placé en tête pour mesurer le temps total, authentification comprise.

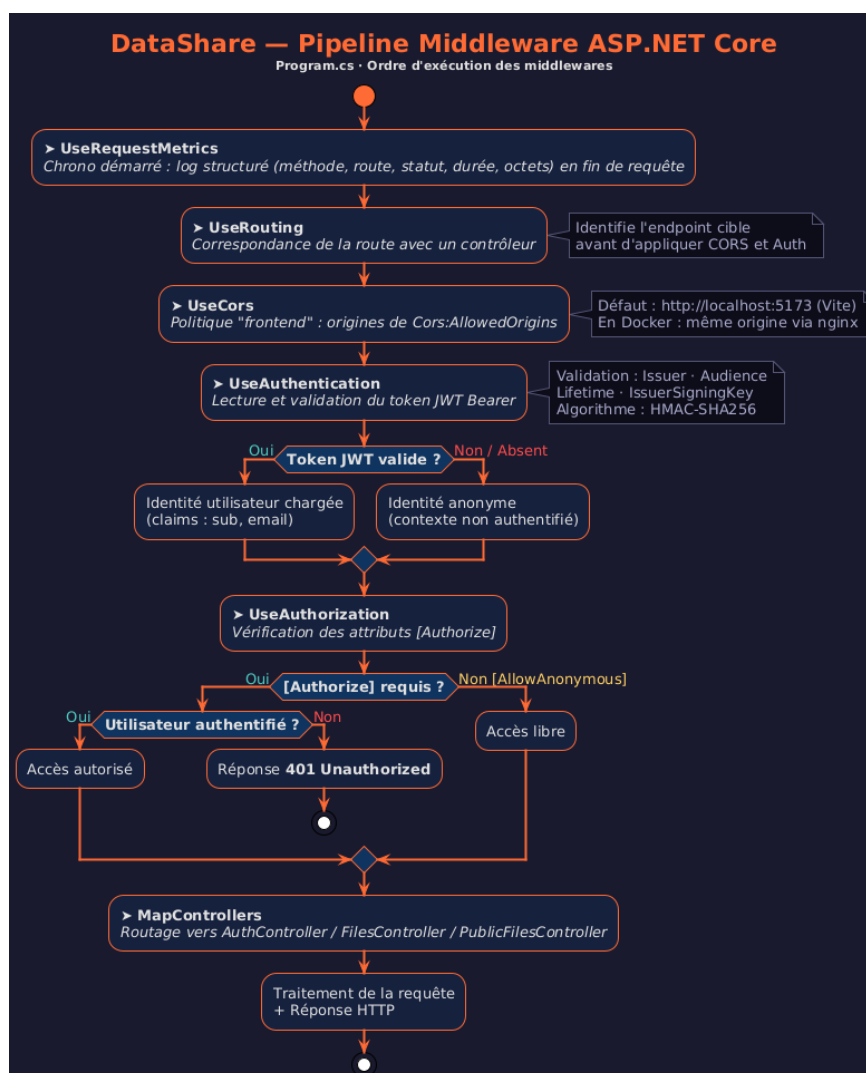


FIGURE 2 — Pipeline middleware ASP.NET Core — ordre d'exécution et bifurcations d'autorisation

2 Choix technologiques justifiés

Les versions ci-dessous sont extraites directement des fichiers `DataShare.Api.csproj` et `package.json` du projet.

2.1 Backend

Technologie	Version	Justification
ASP.NET Core	9.0	Framework web performant, multiplateforme, support natif de l'injection de dépendances et du middleware pipeline.

Technologie	Version	Justification
Entity Framework Core	9.0.1	ORM mature pour .NET, migrations automatiques, protection contre les injections SQL par paramétrisation des requêtes.
Npgsql (EF Core PostgreSQL)	9.0.1	Provider EF Core pour PostgreSQL, support natif des types PostgreSQL (<code>text[]</code>) pour les tags.
PostgreSQL	16	Base relationnelle open source robuste, support des tableaux natifs (<code>text[]</code>), excellentes performances en lecture/écriture.
ASP.NET Core Identity	9.0.1	Gestion complète des utilisateurs (hachage PBKDF2, validation email, contraintes mot de passe) sans code custom.
JWT Bearer Authentication	9.0.1	Authentification stateless adaptée aux SPA, tokens signés HMAC-SHA256, durée de vie configurable (8h).
System.IdentityModel.Tokens.Jwt	8.15.0	Création et validation des tokens JWT côté serveur.
Swashbuckle (Swagger)	10.1.0	Génération automatique de la documentation OpenAPI, UI interactive en développement.

2.2 Tests Backend

Technologie	Version	Justification
xUnit	2.9.2	Framework de tests standard pour .NET, support des théories et faits, parallélisation native.
FluentAssertions	6.12.1	Assertions lisibles en langage naturel (<code>result.Should().Be2000k()</code>).
Moq	4.20.72	Création de mocks pour isoler les unités de code (services, stockage).
Microsoft.AspNetCore.Mvc.Testing	9.0.0	<code>WebApplicationFactory</code> pour les tests d'intégration HTTP sans serveur réel.
EF Core InMemory	9.0.0	Base de données en mémoire pour les tests d'intégration, remplacement transparent de PostgreSQL.
Coverlet	6.0.4	Collecte de couverture de code au format Cobertura, intégré à <code>dotnet test</code> .
ReportGenerator	5.5.1	Génération de rapports HTML de couverture depuis les fichiers Cobertura.

2.3 Frontend

Technologie	Version	Justification
Vue 3	3.5.26	Framework réactif avec Composition API, écosystème riche, courbe d'apprentissage accessible.
TypeScript	5.9.3	Typage statique pour la fiabilité du code frontend, autocomplétion IDE améliorée.
Vite	7.3.0	Bundler ultra-rapide (ESBuild + Rolup), HMR instantané, tree-shaking en production.
Pinia	3.0.4	Store centralisé pour Vue 3, API simple et type-safe, remplacement officiel de Vuex.
Vue Router	4.6.4	Routeur officiel, lazy-loading des composants, mode <code>history</code> HTML5.
Axios	1.13.2	Client HTTP avec intercepteurs pour l'injection automatique du token JWT.
Cypress	15.10.0	Tests E2E fiables, interface graphique interactive, assertions chaînées, screenshots automatiques.

2.4 Infrastructure

Technologie	Version	Justification
Docker	–	Conteneurisation de tous les services pour un déploiement reproductible.
Docker Compose	–	Orchestration locale des 3 services (PostgreSQL, API, Frontend) en une commande.
Nginx	alpine	Serveur de fichiers statiques léger pour le frontend en production.
Node.js	20+	Runtime pour le build frontend (Vite) et les tests Cypress.

3 Modèle de données

Le modèle de données repose sur deux entités métier et les tables système générées par ASP.NET Core Identity.

3.1 Entité AppUser

`AppUser` hérite de `IdentityUser<Guid>` sans propriétés supplémentaires. Les propriétés héritées sont gérées par Identity :

Propriété	Type C#	Description
Id	Guid	Clé primaire (GUID)
UserName	string	Nom d'utilisateur (= email)
Email	string	Adresse email (unique)
PasswordHash	string	Hash PBKDF2 du mot de passe
SecurityStamp	string	Jeton de sécurité (invalidation sessions)
ConcurrencyStamp	string	Contrôle de concurrence optimiste

3.2 Entité FileItem

Propriété	Type C#	Description
Id	Guid	PK, généré par <code>Guid.NewGuid()</code>
OwnerId	Guid?	FK nullable vers <code>AppUser.Id</code>
Owner	<code>AppUser?</code>	Navigation vers le propriétaire
OriginalFileName	string	Nom original du fichier (<code>[MaxLength(255)]</code>)
StoredFileName	string	Nom sur disque (GUID + extension)
ContentType	string?	Type MIME (<code>[MaxLength(255)]</code>)
SizeBytes	long	Taille en octets
Token	string	Token de partage unique (<code>[MaxLength(128)]</code> , index unique)
CreatedAt	<code>DateTimeOffset</code>	Date de création
ExpiresAt	<code>DateTimeOffset</code>	Date d'expiration
PasswordHash	string?	Hash du mot de passe de protection
Tags	string[]	Tags (PostgreSQL <code>text[]</code>)
IsExpired	bool	Propriété calculée (non mappée)
IsPasswordProtected	bool	Propriété calculée (non mappée)

3.3 Relations

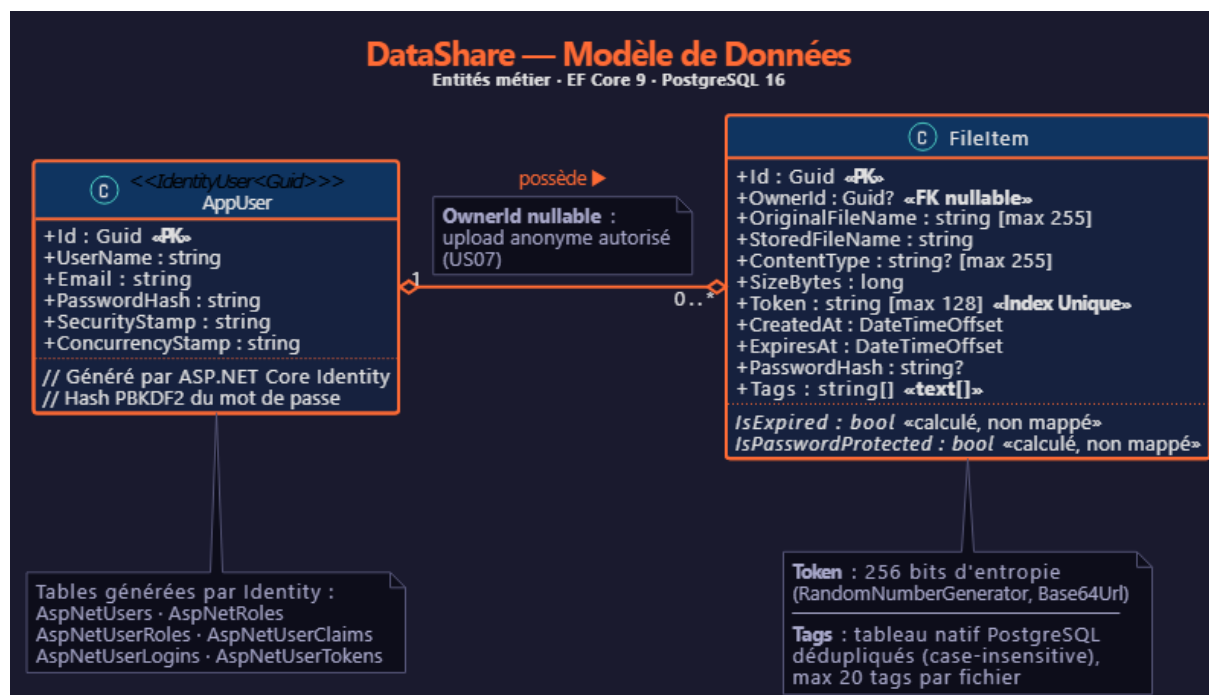


FIGURE 3 — Modèle de données — entités `AppUser` et `FileItem`, relation 1 :N avec `OwnerId` nullable

- **AppUser** (1) → (N) **FileItem** : un utilisateur peut posséder plusieurs fichiers. La relation est optionnelle (`OwnerId` nullable) pour permettre les uploads anonymes (US07).
- Pas de table de jointure : les tags sont stockés comme tableau PostgreSQL natif (`text[]`) directement dans la table `Files`.
- L'index unique sur `Token` garantit l'unicité des liens de partage.

3.4 Configuration Fluent API

La configuration du modèle est définie dans `DataShareDbContext.OnModelCreating` :

```
builder.Entity<FileItem>(e =>
{
    e.HasIndex(x => x.Token).IsUnique();
    e.Property(x => x.Tags).HasColumnType("text []");
});
```

3.5 Historique des migrations

Date	Migration	Description
05/01/2026	InitialAuth	Tables Identity (AspNetUsers, AspNetRoles, etc.)
06/01/2026	AddFiles	Table Files avec FK vers AspNetUsers
16/01/2026	MakeFileOwnerNullable	Préparation pour OwnerId nullable
19/01/2026	MakeOwnerIdNullable	OwnerId rendu nullable (uploads anonymes)

4 Documentation d'API

L'API REST est documentée automatiquement via Swagger (Swashbuckle) accessible en environnement de développement (`dotnet run`) sur `http://localhost:5180/swagger`; le contrat de référence est `docs/openapi.yaml` (OpenAPI 3, validé), doublé de `docs/API_REFERENCE.md`. Le token JWT est obtenu via `POST /api/auth/register` ou `POST /api/auth/login` et expire après **8 heures**.

4.1 Auth — /api/auth

Méth.	Route	Description	Auth	Codes retour
POST	<code>/api/auth/register</code>	Inscription	Non	200 OK, 400 Bad Request
POST	<code>/api/auth/login</code>	Connexion	Non	200 OK, 401 Unauthorized

Corps de requête (register & login) :

```
{
  "email": "alice@example.com",
  "password": "MonMotDePasse1!"
}
```

Réponse succès (200 OK) :

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

4.2 Files (authentifié) — /api/files

Méth.	Route	Description	Auth	Codes retour
POST	<code>/api/files</code>	Upload fichier	Oui	201, 400, 401
GET	<code>/api/files</code>	Lister ses fichiers	Oui	200, 401
GET	<code>/api/files/me</code>	Lister (variante)	Oui	200, 401
GET	<code>/api/files/{id}</code>	Détail d'un fichier	Oui	200, 401, 404
DELETE	<code>/api/files/{id}</code>	Supprimer un fichier	Oui	204, 401, 404

Upload — Corps multipart/form-data :

Champ	Type	Requis	Description
file	binary	Oui	Fichier (max 1 Go)
expiresInDays	integer	Non	Durée 1–7 jours (défaut : 7)
password	string	Non	Mot de passe (min 6 caractères)
tags	string[]	Non	Tags (max 20, dédupliqués)

Réponse upload (201 Created) :

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "originalFileName": "rapport.pdf",
  "sizeBytes": 204800,
  "contentType": "application/pdf",
  "createdAt": "2026-03-10T14:00:00Z",
  "expiresAt": "2026-03-17T14:00:00Z",
  "token": "abc123XYZ_urlsaf3_base64",
  "passwordRequired": false,
  "tags": ["projet", "2026"]
}
```

Réponse listing (200 OK) :

```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "originalFileName": "rapport.pdf",
    "sizeBytes": 204800,
    "contentType": "application/pdf",
    "createdAt": "2026-03-10T14:00:00Z",
    "expiresAt": "2026-03-17T14:00:00Z",
    "token": "abc123XYZ_urlsaf3_base64",
    "passwordRequired": false,
    "tags": ["projet"],
    "isExpired": false
  }
]
```

Paramètre de filtre : ?status=all|active|expired (défaut : all).

Extensions interdites : .exe, .bat, .cmd, .com, .msi, .scr, .ps1.

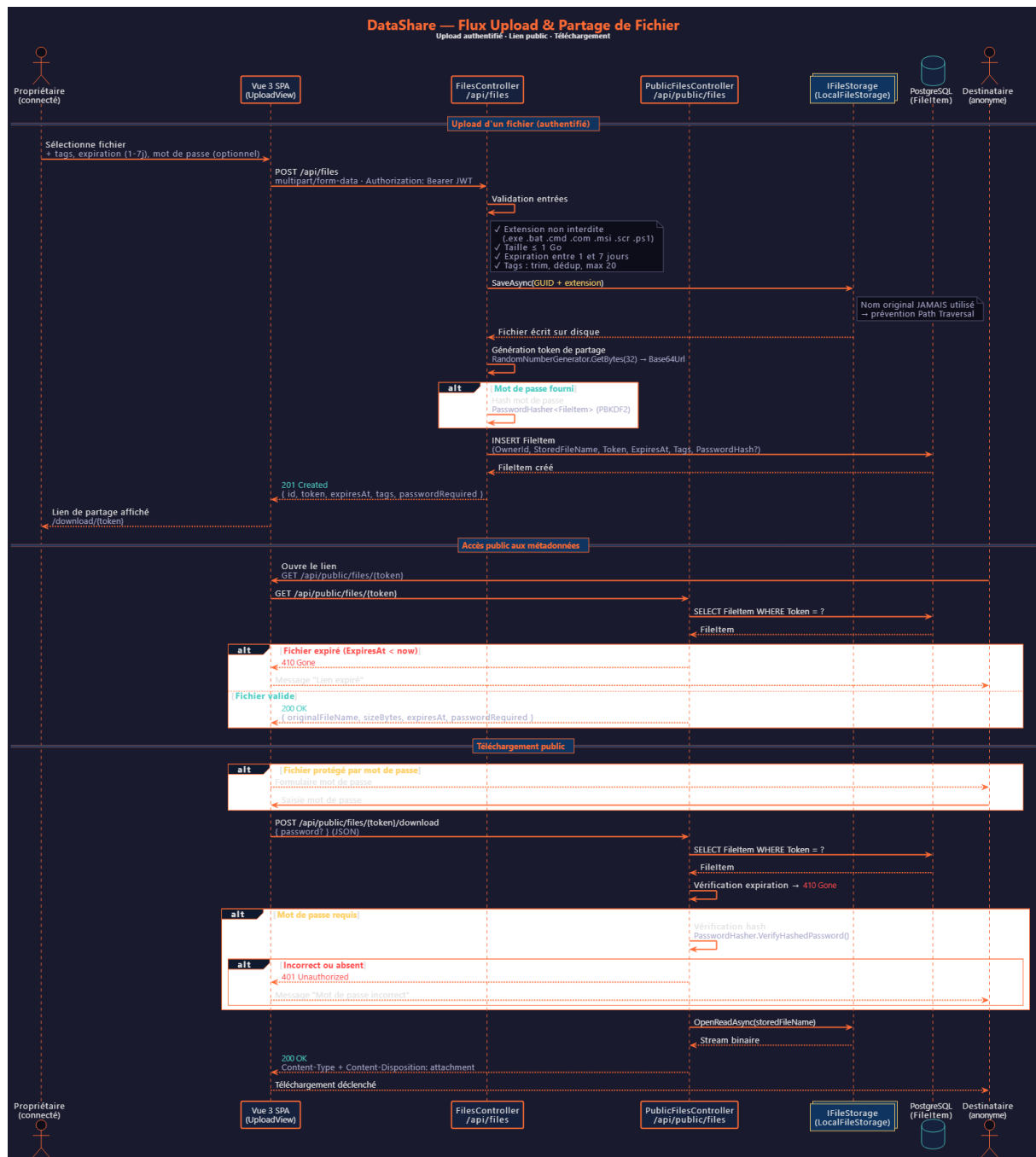


FIGURE 4 – Flux upload et partage — cycle complet de l’upload authentifié au téléchargement public

4.3 Liens publics — /api/public/files

Méth.	Route	Description	Auth	Codes retour
POST	/api/public/files	Upload anonyme	Non*	201, 400, 403
GET	/api/public/files/{token}	Métadonnées	Non	200, 404, 410
POST	/api/public/files/{token}/download	Téléchargement	Non	200, 401, 404, 410

* L'upload anonyme est interdit si l'utilisateur est déjà authentifié (403 Forbidden).

Téléchargement — Corps JSON (optionnel si non protégé) :

```
{
  "password": "monMotDePasse"
}
```

Réponse succès : Stream binaire avec headers `Content-Type` et `Content-Disposition`: `attachment`.

Codes spécifiques :

- **410 Gone** : le lien est expiré.
- **401 Unauthorized** : mot de passe requis mais absent ou incorrect.

5 Sécurité et gestion des accès

La sécurité de DataShare repose sur une approche de défense en profondeur (Defense in Depth), où chaque couche de l'application implémente ses propres mécanismes de protection.

5.1 Authentification JWT

L'authentification utilise exclusivement des JSON Web Tokens (JWT), rendant le serveur stateless (aucune session côté serveur). La configuration est définie dans `Program.cs` :

- **Algorithme** : HMAC-SHA256 (`SecurityAlgorithms.HmacSha256`)
- **Durée de vie** : 8 heures (`DateTime.UtcNow.AddHours(8)`)
- **Validation** : Issuer, Audience, Lifetime et IssuerSigningKey sont tous validés
- **Claims** : sub (ID utilisateur) et email
- **Clé** : lue dans la configuration (`Jwt:Key`, variable d'environnement `Jwt__Key` ou `user-secrets`). L'API **refuse de démarrer** si la clé est absente ou fait moins de 32 caractères : aucune clé de repli n'est codée en dur.
- **Échec de connexion** : réponse 401 identique que l'email soit inconnu ou le mot de passe faux (pas d'énumération de comptes).

Les contrôleurs protégés utilisent l'attribut `[Authorize]` au niveau de la classe (`FilesController`). L'identité de l'utilisateur est extraite du claim `NameIdentifier` pour filtrer l'accès aux fichiers.

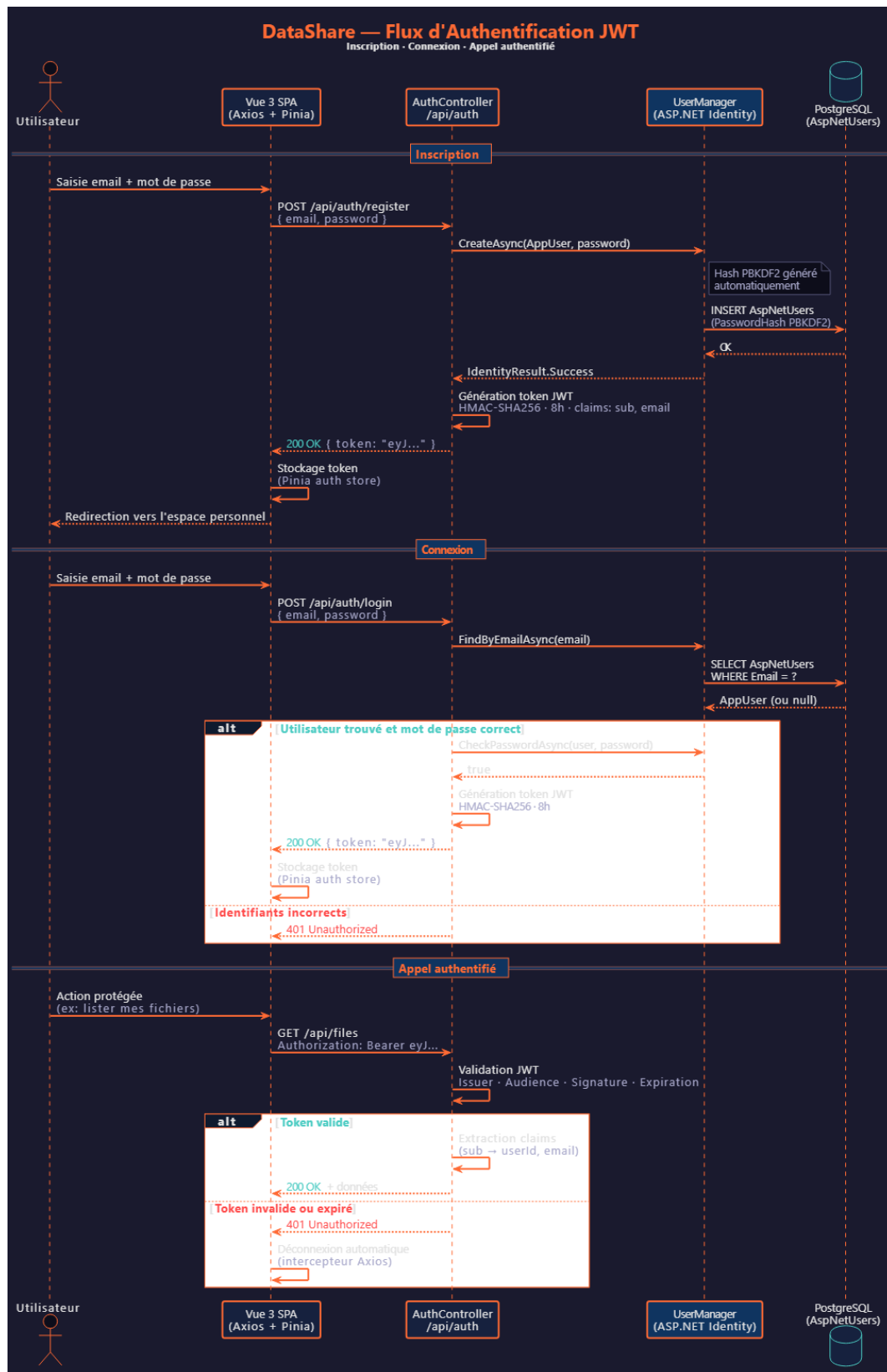


FIGURE 5 – Flux d'authentification JWT — inscription, connexion et appel authentifié

5.2 Hachage des mots de passe

- **Mots de passe utilisateur** : hachés par ASP.NET Core Identity via **PBKDF2** (implémentation standard `PasswordHasher<TUser>`), rendant les attaques par Rainbow Table impraticables.
- **Mots de passe de fichiers** : hachés également via `PasswordHasher<FileItem>` avec le même algorithme PBKDF2.
- **Contraintes** : minimum 8 caractères pour les comptes utilisateur, minimum 6 caractères pour la protection des fichiers.

5.3 Validation des entrées

- **Taille des fichiers** : limitée à 1 Go via `[RequestSizeLimit]` et `FormOptions.MultipartBodyLengthLimit`.
- **Extensions interdites** : les exécutables (`.exe`, `.bat`, `.cmd`, `.com`, `.msi`, `.scr`, `.ps1`) sont rejetés au niveau du contrôleur.
- **Durée d'expiration** : validée entre 1 et 7 jours.
- **Tags** : nettoyés (`trim`), dédoublés (`case-insensitive`), limités à 20 par fichier.
- **Data Annotations** : `[Required]`, `[MaxLength(255)]` et `[MaxLength(128)]` sur les propriétés du modèle `FileItem`.

5.4 Protection des fichiers

- **Sanitization des noms** : aucun fichier n'est stocké avec son nom d'origine. Il est remplacé par un GUID aléatoire (`Guid.NewGuid()`) pour empêcher les attaques de type Path Traversal.
- **Isolation logique** : l'identité vient uniquement du token (`User.FindFirstValue(ClaimTypes.NameIdentifier)`) et chaque requête privée filtre sur `OwnerId == userId` ; un fichier d'un autre utilisateur répond 404 (et non 403) pour ne pas révéler son existence. Ces règles sont vérifiées par les tests d'intégration `SecurityTests` (lecture et suppression croisées, `/files/me` isolé par utilisateur, endpoints privés sans token, métadonnées publiques sans `ownerId` ni token).
- **Token de partage** : généré cryptographiquement via `RandomNumberGenerator.GetBytes(32)` encodé en `Base64Url` (256 bits d'entropie).

5.5 Anti-injection SQL

L'utilisation stricte d'Entity Framework Core (ORM) paramètre toutes les requêtes, neutralisant les injections SQL classiques. Aucune requête SQL brute n'est utilisée dans le projet.

5.6 CORS

La politique CORS nommée "frontend" (`Program.cs`) n'accepte que les origines listées dans `Cors:AllowedOrigins` (par défaut `http://localhost:5173`, le serveur de développement Vite ; surchargeable par `CORS_ALLOWED_ORIGINS` dans `.env`). L'en-tête `Content-Disposition` est exposé pour que le front lise le nom du fichier téléchargé. En Docker, nginx sert le front et proxifie `/api` en même origine : CORS n'entre pas en jeu.

5.7 Transport et journalisation

Le prototype est servi en HTTP sur `localhost` ; HTTPS n'est pas activé dans le code. En production, la terminaison TLS est prévue sur le reverse proxy nginx (certificat Let's Encrypt), pratique standard pour une API conteneurisée. Chaque requête produit une ligne de log structurée (méthode, route, statut, durée, volume) et les refus de mot de passe sur un fichier protégé sont tracés en **Warning** ; aucun mot de passe ni token complet n'est journalisé (voir section 6.4.3).

5.8 Gestion des secrets

- **Développement** : User Secrets .NET (`secrets.json`). Les clés JWT et chaînes de connexion ne sont pas commitées dans Git. Le fichier `.env.example` fournit un template.
- **Production** : les secrets sont injectés via des variables d'environnement (Docker Compose ou CI/CD).

5.9 Améliorations futures

- HTTPS / HSTS : terminaison TLS sur nginx puis `UseHttpsRedirection()` et `UseHsts()` derrière `ForwardedHeaders`.
- Rate Limiting (`AddRateLimiter`, natif .NET) sur `/auth/login` et le téléchargement protégé (prévention brute force).
- Refresh Tokens pour réduire la durée de vie des access tokens (8 h aujourd'hui).
- Liste blanche d'extensions et vérification du type réel (signature) en complément de la liste noire actuelle.
- En-têtes de sécurité (CSP, `X-Content-Type-Options`) côté nginx.
- Scan antivirus des uploads (ClamAV via `nClam`) et chiffrement au repos (implémentation `IFileStorage` dédiée).

6 Qualité, tests et maintenance

6.1 Stratégie de tests

L'approche qualité repose sur la **pyramide des tests**, privilégiant une base solide de tests unitaires et d'intégration pour garantir la robustesse du backend .NET.

Tests unitaires

Validation de la logique métier isolée (services) via des mocks (Moq). Exemple : `ExpiredFilesCleanupServiceTests` valide la purge des fichiers expirés sans dépendance au système de fichiers réel grâce à un mock Moq de `IFileStorage`.

Tests d'intégration

Validation des endpoints API dans un environnement complet via `WebApplicationFactory` et une base de données en mémoire (EF Core InMemory) ; le stockage passe par le vrai `LocalFileStorage` dans un dossier ignoré par Git. Authentification simulée par un `TestAuthHandler` personnalisé, qui accepte un en-tête `X-Test-UserId` pour incarner un second utilisateur. 10 fichiers, **39 tests**, couvrent : authentification (dont email dupliqué, mot de passe court, email inconnu), upload, listing (`/files`

et `/files/me` avec filtre `status`), détail, suppression, cas limites, isolation entre utilisateurs, lien expiré (410) et endpoints publics.

Tests E2E (Cypress)

5 scénarios end-to-end exécutés dans un vrai navigateur, validant les flux utilisateur complets côté frontend.

Stack technique de tests : xUnit 2.9.2, FluentAssertions 6.12.1, Moq 4.20.72, Coverlet 6.0.4, Cypress 15.21. Total : **41 tests** backend (2 unitaires + 39 d'intégration) et 5 scénarios Cypress. Le script `scripts/verify.ps1` enchaîne tests, couverture, build front et audits.

6.2 Plan de tests

Module	Fonctionnalité	Type	Résultat attendu	at-	Critères
Auth	Inscription	Nominal	200 + JWT		Token valide retourné
Auth	Connexion	Nominal	200 + JWT		Token valide retourné
Auth	Connexion	Erreur	401		Pas de token
Files	Upload privé	Nominal	201 + JSON		Fichier accessible
Files	Upload privé	Erreur	400		Fichier absent ou trop gros
Files	Listing	Nominal	200 + liste		Fichiers de l'utilisateur
Files	Download	Nominal	200 + stream		Contenu identique
Files	Download	Erreur	404		ID inexistant
Files	Suppression	Nominal	204		Fichier physique supprimé
Files	Suppression	Erreur	404		Fichier introuvable
Public	Upload anonyme	Nominal	201 + URL		URL fonctionnelle
Public	Download	Nominal	200 + stream		Sans authentification
Core	Stockage local	Unitaire	Fichier créé		Présent sur disque

Tests E2E Cypress :

Fichier	Scénario	Critères
<code>file-lifecycle.cy.ts</code>	Upload, téléchargement, suppression	Cycle complet validé
<code>tags.cy.ts</code>	Ajout et filtrage par tags	Tags visibles et filtrables
<code>password-protection.cy.ts</code>	Protection par mot de passe	Accès refusé/autorisé
<code>limits.cy.ts</code>	Limites de taille/type	Rejet avec message d'erreur
<code>full-scenario.cy.ts</code>	Parcours utilisateur complet	Inscription → upload → partage

6.3 Couverture de code

La couverture est mesurée via Coverlet intégré à `dotnet test`. Le seuil cible est de 80% minimum sur les lignes du code métier (hors migrations EF Core).

Métrique	Résultat	Objectif
Couverture en lignes	83,2%	$\geq 80\%$
Couverture en branches	62%	$\geq 50\%$
Couche Controllers (Auth)	100%	–
Couche Controllers (Files)	74%	–
Couche Controllers (Public)	90,7%	–
Couche Services	75,5–100%	–

Mesure de référence du 02/08/2026 (339 lignes couvertes sur 407, hors migrations). L'endpoint `GET /api/files/me`, alors non couvert, l'est depuis l'ajout de `SecurityTests` (13/09/2026). Le rapport HTML est généré dans `backend/TestResults/CoverageReport/index.html`.

Commandes :

```
dotnet test --collect:"XPlat Code Coverage" --settings coverlet.
runsettings
reportgenerator -reports:"**/coverage.cobertura.xml" \
-targetdir:"coverage-report" -reporttypes:Html
```

6.4 Performance

6.4.1 Budget de performance

Indicateur	Cible
API — P95 upload 100 Ko	< 500 ms
API — P95 connexion (PBKDF2)	< 500 ms
API — taux d'erreur sous charge	< 1%
Front — bundle JS initial (gzip)	< 100 Ko
Front — Lighthouse Performance	≥ 90
Front — LCP / CLS / TBT	< 2,5 s / < 0,1 / < 200 ms

6.4.2 Résultats du test de charge k6

Test réalisé le 17 février 2026 avec 20 utilisateurs virtuels pendant 30 secondes :

Métrique	Résultat	Objectif	Statut
Itérations totales	500 uploads	> 100	Passé

Métrique	Résultat	Objectif	Statut
Requêtes/seconde	~32/s	> 10/s	Passé
Temps moyen	107,9 ms	< 500 ms	Passé
P95	125,19 ms	< 2000 ms	Passé
Taux d'erreur	0,00%	< 1%	Passé
Données transférées	52 Mo	–	Info

Analyse : à 20 utilisateurs simultanés, l'upload complet (validation JWT, écriture disque, insertion PostgreSQL) reste autour de 100 ms sans dégradation sur 30 s ; les I/O sont asynchrones (`CopyToAsync`, `SaveChangesAsync`), la limite réelle est le disque et le réseau, pas le code. Les scripts `perf/k6-upload-test.js` et `perf/k6-login-test.js` créent leur propre compte de test (`setup()`) et acceptent `BASE_URL`, `VUS`, `DURATION`, `FILE_KB` en variables d'environnement.

6.4.3 Logs structurés et métriques clés

L'API journalise des événements structurés (placeholders nommés capturés comme propriétés par `ILogger`) ; hors développement (conteneur Docker) la sortie est en **JSON**, **une ligne par événement** (`AddJsonConsole`), directement exploitable par un collecteur (Seq, Loki, ELK) ou par `grep`.

Source	Événement	Propriétés
RequestMetricsMiddleware	chaque requête HTTP	Method, Path, StatusCode, ElapsedMs, RequestBytes, ResponseBytes (niveau Warning si 4xx, Error si 5xx)
FilesController, PublicFilesController	upload réussi	FileId, SizeBytes, ContentType, UserId, ExpiresAt, PasswordProtected, TagCount
PublicFilesController	téléchargement / mot de passe refusé	FileId, SizeBytes, ContentType
FilesController	suppression	FileId, SizeBytes, UserId
ExpiredFilesCleanupService	purge quotidienne	Count, erreurs de stockage

Exploitation : P95 des uploads et volume transféré depuis les logs Docker

```
docker compose logs api | grep '"Path":"/api/files"' | grep '"Method":"POST"' \
| grep -o '"ElapsedMs":[0-9.]*' | cut -d: -f2 | sort -n \
| awk '{a[NR]=$1} END {print "n="NR, "p95="a[int(NR*0.95)]" ms"}'
docker compose logs api | grep -o '"SizeBytes":[0-9.]*' | cut -d: -f2 \
| awk '{s+=$1} END {print s" octets"}'
```

6.4.4 Budget frontend (Lighthouse)

Métrique	Budget	Justification
Bundle Size (Gzip)	< 350 Ko	Chargement rapide sur mobile
FCP (First Contentful Paint)	< 1,5 s	Affichage rapide
LCP (Largest Contentful Paint)	< 2,5 s	Contenu principal visible
CLS (Cumulative Layout Shift)	< 0,1	Stabilité visuelle
Score Performance	≥ 90	–
Score Accessibility	≥ 90	–

Optimisations mises en place : bundling Vite (tree-shaking), lazy-loading des routes Vue Router, CSS minifié en production.

6.4.5 Résultats Lighthouse (audit du 31/03/2026)

Catégorie	Score	Objectif	Statut
Performance	96	≥ 90	Passé
Accessibility	76	≥ 90	À améliorer
Best Practices	100	≥ 90	Passé
SEO	82	≥ 80	Passé

Métriques détaillées : FCP 2,1s, LCP 2,3s, TBT 0 ms, CLS 0,001, Speed Index 2,1s.

Analyse : Le score Accessibility (76) est en dessous de l'objectif — les améliorations prioritaires concernent les contrastes et attributs ARIA. Le FCP (2,1 s) dépasse légèrement le budget de 1,5s, optimisable via le préchargement des fonts.

6.5 Maintenance

6.5.1 Gestion des services (Docker)

```
docker compose up -d --build    # Lancer (ou scripts/deploy.ps1 |
  deploy.sh)
docker compose ps                # Etat et healthchecks (db, api)
docker compose logs -f api       # Logs structures JSON de l'API
docker compose down             # Arrêter (donnees et fichiers
  conservees)
docker compose down -v          # Reset complet (base + fichiers
  uploades)
```

Les fichiers uploadés sont conservés dans le volume `datashare_uploads`, la base dans `datashare_pg`. Les migrations EF Core sont appliquées automatiquement au démarrage de l'API.

6.5.2 Sauvegarde et restauration de la base

```
# Sauvegarde
docker exec -t oc-p4-datashare-postgres \
  pg_dump -U datashare datashare > backup_$(date +%F).sql

# Restauration
docker exec -i oc-p4-datashare-postgres \
  psql -U datashare datashare < backup_2026-01-15.sql
```

Scripts équivalents Windows : `scripts/db-backup.ps1` et `scripts/db-restore.ps1`. Base et volume de fichiers doivent être sauvegardés ensemble (les métadonnées référencent les noms stockés). Fréquence recommandée : quotidienne en production, avant toute mise à jour majeure.

6.5.3 Mise à jour des dépendances

- **Backend** : `dotnet list package -outdated` (mensuel), `audit dotnet list package -vulnerable -include-transitive` à chaque livraison, correctifs de sécurité immédiats, vérification par `dotnet test`.
- **Frontend** : `npm audit` à chaque livraison, `npm outdated` mensuel, vérification par `npm run build` (type-check) et `npm run lint`; jamais `npm audit fix -force` sans revue.
- **Majeures** (.NET, Vite, Vue, PostgreSQL) : 1 à 2 fois par an, sur branche dédiée, avec guide de migration et suite de tests complète; changement de majeure PostgreSQL = `pg_dump` / restauration, jamais sur un volume existant.
- **Cas rencontrés** : (08/2026) mise à jour de sécurité d'`axios` détectée par le type-check et corrigée en une ligne; (09/2026) `npm audit fix` sur 13 avis transitifs (outils de build et Cypress) sans impact fonctionnel. Détail et décisions dans `SECURITY.md`.

6.5.4 Procédure de correction de bugs

1. Reproduire le bug (écrire un test qui échoue).
2. Isoler : backend, frontend ou base de données.
3. Corriger sur une branche dédiée (`fix/description-courte`).
4. Vérifier : `dotnet test + npx cypress run`.
5. Merger après revue et tests verts.

6.5.5 Nettoyage des fichiers expirés

Les fichiers ont une durée de vie maximale de 7 jours. Le nettoyage est **automatique** : le service `ExpiredFilesCleanupService` s'exécute au démarrage de l'API puis toutes les 24 heures. Il supprime les fichiers physiques et les entrées en base de données.

6.5.6 Checklist de mise en production

- ☐ `.env` renseigné : mot de passe PostgreSQL fort, `JWT_KEY` aléatoire ≥ 32 caractères, `CORS_ALLOWED_ORIGINS`
- ☐ `ASPNETCORE_ENVIRONMENT=Production` (Swagger désactivé, logs JSON)
- ☐ HTTPS terminé sur le reverse proxy

- ☐ Backup base de données effectué
- ☐ `dotnet test` — tous les tests passent
- ☐ `npm audit` — aucune vulnérabilité critique
- ☐ Fichiers statiques frontend buildés (`npm run build`)

7 Processus d'installation et d'exécution

7.1 Prérequis

Outil	Version	Notes
.NET SDK	9.0	Compilation et exécution du backend
Node.js	20+	Build frontend (Vite) et tests Cypress
PostgreSQL	16	Via Docker ou installation locale
Docker & Compose	–	Optionnel, recommandé pour l'installation simplifiée

7.2 Installation manuelle

1. Cloner le projet

```
git clone https://github.com/GadZouil/OC-P4-DataShare.git
cd OC-P4-DataShare
```

2. Base de données PostgreSQL

```
docker-compose up -d db
```

3. Backend

```
cd backend/DataShare.Api
dotnet user-secrets set "ConnectionStrings:Default" \
  "Host=127.0.0.1;Port=5433;Database=datashare;Username=datashare;
  Password=datashare"
dotnet run
# API disponible sur http://localhost:5180 (migrations appliquées
  au démarrage)
# Swagger UI sur http://localhost:5180/swagger
```

4. Frontend

```
cd frontend/datashare-front
npm install
npm run dev
# Application disponible sur http://localhost:5173
```

7.3 Installation Docker (tout-en-un)

```
cp .env.example .env # optionnel : secrets (valeurs de
demo par défaut)
docker compose up -d --build # ou : ./scripts/deploy.sh | .\
scripts\deploy.ps1
```

Cette commande lance les trois services :

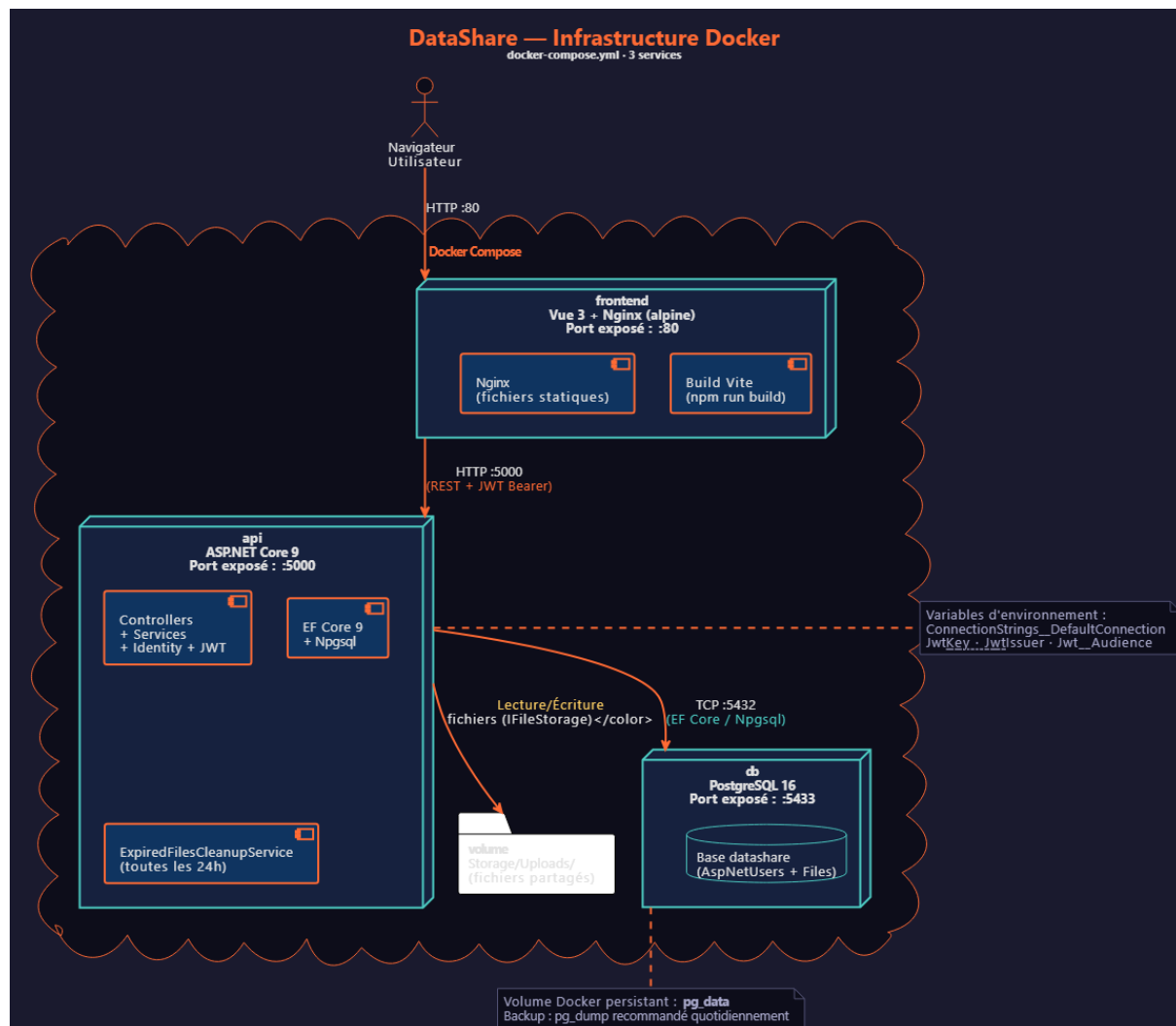


FIGURE 6 – Infrastructure Docker — trois services orchestrés par Docker Compose

- **db** — PostgreSQL 16 sur le port 5433
- **api** — API ASP.NET Core sur le port 5000
- **frontend** — Vue 3 (Nginx) sur le port 80

7.4 Variables d'environnement

Le fichier `.env.example` fournit le template des variables à configurer :

Variable	Exemple	Description
POSTGRES_DB / _USER / _PASSWORD	datashare / datashare / (secret)	Base, utilisateur et mot de passe PostgreSQL
JWT_KEY	(32+ caractères)	Clé de signature JWT (l'API refuse de démarrer sinon)
JWT_ISSUER / JWT_AUDIENCE	DataShareApi / DataShareClient	Émetteur et audience des tokens
CORS_ALLOWED_ORIGINS	http://localhost:5173	Origine autorisée (front hors nginx)
VITE_API_URL	/api	URL de l'API pour le build du front

Configuration complète (appsettings.json) :

Clé	Description
ConnectionStrings:Default	Chaîne de connexion PostgreSQL
Jwt:Issuer	Émetteur du token (défaut : DataShare)
Jwt:Audience	Audience du token (défaut : DataShare)
Jwt:Key	Clé secrète HMAC-SHA256 (min 32 caractères)

`docker-compose.yml` lit ces variables dans `.env` et les injecte dans la section `environment` du service `api` (Section_`Cle` est lu par ASP.NET Core comme `Section:Cle`) :

```
api:
  environment:
    ConnectionStrings__Default: "Host=db;Port=5432;Database=${
      POSTGRES_DB:-datashare};..."
    Jwt__Key: "${JWT_KEY:-change-me-in-production-secret-key-32chars}"
    Jwt__Issuer: "${JWT_ISSUER:-DataShareApi}"
    Jwt__Audience: "${JWT_AUDIENCE:-DataShareClient}"
    Cors__AllowedOrigins__0: "${CORS_ALLOWED_ORIGINS:-http://localhost:5173}"
```

8 Utilisation de l'IA dans le développement

8.1 Posture adoptée

L'IA a été utilisée selon une approche “**assignation de tâche à un développeur junior**” pour la tâche principale (US08 — Gestion des tags), puis en **binômeage ponctuel** pour le reste du projet.

Phase	Outil	Usage
Début de projet	ChatGPT	Aide générale, premières questions d'architecture
Questions complexes	Claude Opus (claude.ai)	Vision globale, décisions structurantes
Développement	Claude Opus & Sonnet (Cursor)	Opus pour le contexte complet du codebase, Sonnet pour les zones ciblées

L'évolution a été naturelle : ChatGPT pour démarrer, puis Claude Opus en ligne pour les réflexions d'architecture, et enfin Cursor avec le codebase chargé pour le développement concret. Le passage à Cursor a permis à l'IA d'avoir le contexte projet complet, rendant les réponses bien plus pertinentes.

8.2 Tâche principale : US08 — Gestion des tags (frontend)

8.2.1 Contexte

Le backend supportait déjà les tags (`string[]` à l'upload, renvoyés dans les réponses API). L'US08 consistait à implémenter toute la partie frontend : champ de saisie de tags (chips) sur la page Upload, affichage des tags sur la page “Mon espace”, et filtre local par tag (case-insensitive, contains).

8.2.2 Fichiers modifiés par l'IA

- `src/views/UploadView.vue` — Champ de saisie de tags avec chips
- `src/views/MeView.vue` — Affichage et filtrage par tags
- `src/styles/datashare.css` — Styles des chips et du filtre
- (`src/api/files.ts` était autorisé « si nécessaire » : l'envoi des tags en `FormData` existait déjà, aucune modification)

Le prompt complet est disponible dans le fichier `IA_Instructions.txt` à la racine du projet. Traçabilité Git : commit `56f411d feat(ai): add tags UI and filtering` (code produit par l'IA) puis `a8c23fc fix(tags): show clear UI errors...` (revue humaine).

8.3 Supervision et corrections humaines

Après réception du code généré par l'IA, les corrections suivantes ont été apportées manuellement :

Problème détecté	Correction
Messages d'erreur UI absents pour tags invalides	Ajout de feedback visuel pour chaque cas d'erreur (vide, trop long, doublon)
Validation insuffisante côté UI	Renforcement des contrôles (trim, longueur max 24 caractères, unicité case-insensitive)

Processus de revue :

1. Lecture complète du diff généré par l'IA.

2. Test manuel des cas limites (tag vide, tag > 24 caractères, doublon exact, doublon casse différente).
3. Identification des lacunes UX (aucun message d'erreur pour l'utilisateur).
4. Correction et commit séparé pour traçabilité.

Points de vigilance vérifiés : le back reste l'autorité (normalisation et déduplication des tags aussi côté API, 20 tags max) ; aucune nouvelle dépendance ; styles ajoutés dans `datashare.css` avec le préfixe `ds-` existant ; logique isolée dans `addTag` / `removeTag` ; conformité à la charte Figma.

8.4 Autres usages de l'IA

Usage	Détail
Audit de code	Revue de sécurité, détection de dépendances inutiles, vérification de cohérence
Debugging	Résolution d'erreurs CORS, JWT, configuration Docker
Architecture	Conseils sur l'organisation Services/Models/Controllers
Recherche technique	Bonnes pratiques .NET, Vue 3, PostgreSQL — remplace efficacement Google/Stack Overflow
Documentation	Assistance rédaction des fichiers <code>.md</code> techniques
Revue finale avant soutenance (09/2026)	Audit de cohérence code ↔ documentation (OpenAPI périmé, <code>SECURITY.md</code> décrivant des mécanismes absents du code), propositions de tests de sécurité et de logs structurés, relecture du support ; chaque proposition relue, exécutée (<code>dotnet test</code> , <code>npm run build</code>) et validée avant commit

8.5 Apports et limites constatés

8.5.1 Apports

- **Gain de temps :** US08 complète (~200 lignes, 4 fichiers) produite en ~15 minutes contre ~2 heures estimées manuellement.
- **Audit efficace :** détection rapide de problèmes de sécurité et de dépendances inutiles.
- **Recherche contextuelle :** réponses adaptées au projet, pas de réponses génériques.
- **Cohérence :** code généré respectant le style existant du projet.

8.5.2 Limites

- **UX négligée :** l'IA n'a pas pensé aux messages d'erreur utilisateur.
- **Pas de tests spontanés :** aucun test proposé pour le code généré.
- **Supervision indispensable :** sans revue humaine, les cas limites auraient été livrés sans feedback utilisateur.

8.6 Conclusion sur l'utilisation de l'IA

L'IA s'est révélée efficace comme outil de productivité à tous les niveaux du projet. L'approche “junior supervisé” pour l'US08 a bien fonctionné, et l'usage en binôme pour le reste (audit, debug, recherche) a significativement accéléré le développement. La supervision humaine reste indispensable pour la qualité UX et la couverture de tests.